# UNIVERSITY OF THE PEOPLE

**CS 2301-01 - AY2027-T1**

**Unit 1 Assignment**

Submitted by: Mubashir Ali

Instructor: Joseph Ego

Date: September 18, 2026

# Deadlock Prevention and Avoidance in a High-Performance Computing Cluster

## Introduction

A high-performance computing (HPC) cluster must manage many tasks concurrently while sharing limited resources such as CPUs, memory, and storage. Because several tasks may request different resources at the same time, improper resource allocation can create a deadlock. A deadlock occurs when processes become permanently blocked because each process is waiting for a resource held by another process. In a large computing cluster, this can reduce resource utilization, delay jobs, and affect the reliability of the entire system. Therefore, the resource management system needs an effective strategy for preventing or avoiding deadlocks.

Two practical deadlock prevention techniques for the proposed HPC cluster are resource ordering and eliminating the hold-and-wait condition by requiring processes to request resources together before execution. These techniques work by ensuring that at least one of the necessary conditions for deadlock cannot occur. This discussion also compares deadlock prevention with deadlock avoidance and examines the trade-offs between the two approaches.

# 1. Two Deadlock Prevention Techniques

## Technique 1: Resource Ordering

Resource ordering is a deadlock prevention technique in which all resource types are assigned a fixed global order. A task must request resources according to this predefined order and cannot request a lower-ordered resource after already acquiring a higher-ordered resource. Hailperin (2019) explains resource ordering as a practical method for preventing deadlock because processes acquire resources in an agreed-upon order.

For the HPC cluster, the administrator could establish an order such as:

**CPU → Memory → Storage**

This does not mean that every task must always use all three resources. Instead, if a task requires multiple resources, it must request them according to the established order. For example, a task that needs CPU and storage would request the CPU resource before requesting storage. Another task could not hold storage while waiting for CPU.

This technique directly prevents the circular-wait condition. Consider two tasks, Task A and Task B. Without resource ordering, Task A might hold CPU resources while waiting for storage, while Task B holds storage resources while waiting for CPU. The two tasks would wait indefinitely for each other. With a global resource order, this pattern is prohibited because a task holding storage cannot request CPU if CPU has a lower order. Consequently, a circular chain of resource requests cannot form.

Resource ordering is particularly suitable for an HPC cluster because the resource manager can enforce the policy centrally. The ordering can be applied consistently to CPU allocations, memory reservations, storage resources, and other shared resources. By preventing circular wait at the design level, the system can guarantee that this particular cause of deadlock does not occur (Hailperin, 2019).

## Technique 2: Eliminating Hold and Wait

A second prevention technique is to eliminate the hold-and-wait condition. Under this approach, a task must request all the resources it will need before it begins execution. The task is allocated the required resources only when all of them are available. If the complete set is unavailable, the task waits without holding any of the resources.

For example, suppose an HPC task requires 8 CPU cores, 32 GB of memory, and 500 GB of temporary storage. Under a hold-and-wait prevention policy, the resource manager would not give the task the CPU cores and

memory while making it wait for storage. Instead, the task would wait until all three requirements could be satisfied. Once they are available, the resource manager allocates them together and the task begins execution.

This prevents a task from holding one resource while waiting indefinitely for another. Therefore, one of the necessary conditions for deadlock is removed. For example, if Task A has CPU resources and is waiting for storage, this technique does not allow Task A to retain the CPU while waiting. It must either receive its complete resource set or wait without holding resources.

The main advantage is that the resource manager has a clear and predictable allocation policy. However, this approach can reduce resource utilization because resources may remain reserved for a task even when the task is not actively using every allocated resource. It can also be difficult for workloads whose resource requirements change dynamically during execution.

# 2. How the Techniques Prevent Deadlocks

The two techniques prevent deadlocks in different ways.

Resource ordering prevents circular wait. Every task follows the same resource hierarchy, so a cycle such as Task A waiting for a resource held by Task B while Task B waits for a resource held by Task A cannot be created. Hailperin (2019) demonstrates this principle using ordered resource acquisition: when resources are acquired according to an agreed order, the circular dependency necessary for deadlock cannot form.

Eliminating hold and wait attacks a different deadlock condition. A task cannot hold CPU or memory while waiting for storage because it must obtain its required resource set before execution. Thus, the system does not allow a task to remain in a state where it possesses some resources while waiting for additional resources.

These techniques can therefore be summarized as follows:

| Prevention technique | Condition addressed | Application to HPC cluster |
|---|---|---|
| Resource ordering | Circular wait | Tasks request CPU, memory, storage, and other resources according to a fixed global order |
| Eliminate hold and wait | Hold and wait | Tasks request their required resource set before execution rather than holding partial resources while waiting |

Together, these policies significantly restrict the conditions required for deadlock. The important point is that prevention does not need to identify a deadlock after it occurs. Instead, the resource manager designs allocation rules so that a deadlock condition cannot develop.

# 3. Deadlock Avoidance Compared with Deadlock Prevention

Deadlock prevention and deadlock avoidance have the same overall objective, but they use different strategies.

Deadlock prevention establishes restrictions that guarantee at least one of the necessary deadlock conditions cannot occur. In the HPC cluster, resource ordering can prevent circular wait, while eliminating hold and wait prevents tasks from retaining resources while requesting additional ones. The rules are normally applied regardless of the current resource state.

Deadlock avoidance, in contrast, allows more flexible resource allocation but checks whether each proposed allocation could place the system in an unsafe state. A well-known example is the Banker's algorithm. Before granting a request, the resource manager considers the resources currently available, the resources allocated to each task, and the maximum resources each task may still need. The request is granted only when the resulting state remains safe. Operating System Concepts identifies the Banker's algorithm as a deadlock-avoidance method that evaluates whether resource allocation can maintain a safe state. (Silberschatz et al., 2018).

For the HPC scenario, consider three tasks competing for CPU cores, memory, and storage. A prevention-based manager might require every task to follow a fixed resource order or request all resources in advance. An avoidance-based manager could instead examine the current allocation state and decide whether granting a particular request would still allow all active tasks to eventually complete.

This makes avoidance more dynamic. For example, if a task requests additional memory, the system does not necessarily reject the request simply because another task currently holds resources. Instead, it evaluates whether granting the memory would leave the system in a safe state. If a safe sequence exists in which the remaining tasks can complete and release their resources, the request may be granted.

The key distinction is therefore that prevention changes the rules of resource acquisition, while avoidance evaluates resource requests against the current state of the system. Prevention provides a structural guarantee by restricting resource behavior. Avoidance provides a dynamic guarantee by refusing allocations that would create an unsafe state.

# 4. Trade-Offs Between Prevention and Avoidance

The choice between prevention and avoidance involves important performance and complexity trade-offs.

Deadlock prevention is generally simpler to reason about and implement because its policies are established in advance. Resource ordering, for example, requires the resource manager to maintain a consistent ordering and

enforce it whenever a task requests multiple resources. Once correctly implemented, the system does not need to perform a complex safety analysis for every allocation request. This can reduce runtime overhead and make the system more predictable.

However, prevention can be restrictive. Requiring tasks to obtain all resources before execution may cause resources to remain unused while a task waits for another resource. A task might hold memory even though it is waiting for storage, or the system might refuse to start a job because its complete resource set is temporarily unavailable. This can reduce resource utilization and potentially increase waiting time.

Deadlock avoidance provides greater flexibility because resources can be allocated incrementally. The resource manager evaluates the current state before granting requests. This can improve resource utilization because resources do not necessarily have to be reserved in advance. However, avoidance requires additional information and computation. For example, the Banker's algorithm needs information about current allocations, available resources, and the maximum potential demands of processes. The system must repeatedly perform safety checks when resource requests are made.

This creates additional computational and management overhead. It can also be difficult to apply when HPC workloads cannot accurately predict their maximum resource requirements. If a task's future resource demand is unknown or changes significantly during execution, the assumptions required by avoidance algorithms may become difficult to maintain.

Therefore, the trade-off can be summarized as follows:

| Factor | Deadlock Prevention | Deadlock Avoidance |
|---|---|---|
| Basic strategy | Restrict resource allocation rules | Dynamically evaluate each allocation |
| Flexibility | Lower | Higher |
| Runtime decision-making | Relatively simple | More complex |
| Resource utilization | Can be lower because of restrictions | Can be higher because allocation is dynamic |
| Required information | Resource-ordering or acquisition policy | Current allocation and potential future demands |
| Complexity | Generally simpler | Generally, more computationally complex |
| Guarantee | Prevents deadlock by design | Avoids unsafe states through resource checks |

For a high-performance cluster, the appropriate choice depends on the workload. If resource requirements are predictable and the cluster prioritizes simplicity and predictable behavior, prevention can be effective. If

workloads have variable resource requirements and maximizing resource utilization is important, avoidance can provide greater flexibility, provided that the system can maintain accurate information about resource demands.

# Conclusion

Deadlock is an important resource-management problem in a high-performance computing cluster because multiple tasks may simultaneously compete for CPUs, memory, storage, and other limited resources. Two effective prevention techniques are resource ordering and eliminating hold and wait. Resource ordering prevents circular wait by requiring tasks to acquire resources according to a consistent global order. Eliminating hold and wait prevents tasks from holding partial resource allocations while waiting for additional resources.

Deadlock avoidance differs because it does not impose the same fixed restrictions on every request. Instead, it evaluates whether granting a request would leave the system in a safe state. The Banker's algorithm is a classic example of this approach. Prevention is generally simpler and can have lower decision-making overhead, but it may restrict resource utilization. Avoidance offers greater flexibility but requires more information, state tracking, and runtime computation.

For the proposed HPC cluster, the most important consideration is balancing guaranteed deadlock-free behavior with efficient resource utilization. Understanding these differences allows the resource manager to select an approach that matches the characteristics of the cluster's workloads while maintaining system reliability and performance.

# References

Hailperin, M. (2019). *Operating systems and middleware: Supporting controlled interaction* (Rev. ed. 1.3.1). Gustavus Adolphus College.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley.